

RoboRacer Control: MPPI and Nonlinear MPC

Mathematical Overview

Contents

1	Vehicle Model	3
1.1	States and Inputs	3
1.2	Continuous-Time Dynamics	3
1.3	Constraints	3
2	Raceline and Reference Trajectory	3
2.1	Windowed Closest-Point Search	4
2.2	Lookahead	4
3	MPPI — Model Predictive Path Integral Control	4
3.1	Conceptual Overview	4
3.2	Discrete-Time Dynamics (Euler)	5
3.3	Algorithm	5
3.4	Cost Function	6
3.5	Warm-Start from Raceline Curvature	6
3.6	Velocity Tracking Change	7
4	Nonlinear MPC	7
4.1	Conceptual Overview	7
4.2	Continuous-Time Dynamics	7
4.3	Optimal Control Problem	8
4.4	Cost Function	8
4.5	Reference Heading Unwrapping	9
4.6	Warm-Start from Raceline Curvature	9
4.7	SQP Real-Time Iteration (SQP-RTI)	9
5	PID Baseline (Stanley-style)	10
5.1	Conceptual Overview	10
5.2	Signed Cross-Track Error	10
5.3	Derivative and Integral Terms	10
5.4	Heading Error Term and Stanley Analogue	11
5.5	Parameters	11
5.6	Tuning Procedure	11

6	Throttle Control	11
7	Future Model Improvements	12
7.1	Dynamic Bicycle Model	12
7.2	Pacejka “Magic Formula” Tire Model	13
8	Experimental Results	13
8.1	Metrics	13
8.2	Results	14
8.3	Discussion	14

1 Vehicle Model

Both controllers share the same underlying vehicle model: the **kinematic bicycle model**. It treats the car as a single-track rigid body with no tire slip, making it valid at low-to-moderate speeds where lateral acceleration is small enough that the tires remain near the linear region of their force–slip curves.

1.1 States and Inputs

$$\mathbf{x} = \begin{pmatrix} X \\ Y \\ \psi \\ v \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} \delta \\ a \end{pmatrix}$$

Symbol	Description	Unit
X, Y	Global position	m
ψ	Heading angle (yaw)	rad
v	Longitudinal speed	m s ^{−1}
δ	Front-wheel steering angle	rad
a	Longitudinal acceleration	m s ^{−2}
L	Wheelbase	m

1.2 Continuous-Time Dynamics

$$\dot{X} = v \cos \psi \tag{1}$$

$$\dot{Y} = v \sin \psi \tag{2}$$

$$\dot{\psi} = \frac{v}{L} \tan \delta \tag{3}$$

$$\dot{v} = a \tag{4}$$

Equation (3) is derived from Ackermann geometry: for a bicycle model the instantaneous turning radius is $R = L / \tan \delta$, so the yaw rate is $\dot{\psi} = v / R$.

1.3 Constraints

$$\delta \in [-\delta_{\max}, \delta_{\max}], \quad a \in [a_{\min}, a_{\max}], \quad v \in [0, v_{\max}]$$

with $\delta_{\max} = 0.44 \text{ rad}$, $a_{\min} = -6.15$, $a_{\max} = 6.15 \text{ m s}^{-2}$, $v_{\max} = 15 \text{ m s}^{-1}$, and $L = 0.324 \text{ m}$.

2 Raceline and Reference Trajectory

A pre-computed raceline CSV provides the reference:

$$\mathcal{R} = \{(X_i, Y_i, \psi_i, v_{\text{ref},i}, s_i)\}_{i=0}^{M-1}$$

where s_i is the cumulative arc length at waypoint i .

2.1 Windowed Closest-Point Search

The **closest waypoint index** i^* is the nearest neighbour of the car’s position (X, Y) in the raceline. A pure exhaustive search is correct but fails at *self-intersections* of the raceline: on the figure-8 track the loops cross at a single physical point that maps to two different arc lengths, and an exhaustive search can flip between branches when the car is near the crossing.

To prevent this, after the first call the search is restricted to a window of ± 100 indices around the previous i^* . This forces monotonic progress along the arc and eliminates the crossing ambiguity at the cost of requiring that the car never moves more than one window’s worth of arc length between control steps (at $\Delta t = 25$ ms and 100 indices ≈ 2.9 m of arc, this corresponds to speeds well above $v_{\max}$).

2.2 Lookahead

Lookahead reference points at horizon step k are obtained by projecting k steps forward in arc length:

$$s_{\text{ref}}(k) = s_{i^*} + k \cdot v_{\text{look}} \cdot \Delta t, \quad v_{\text{look}} = \max(\hat{v}, v_{\text{look},\min})$$

where $\hat{v}$ is the current (estimated) car speed and $v_{\text{look},\min} = 0.8 \text{ m s}^{-1}$ is a floor that keeps the lookahead non-degenerate when stationary. Looking the reference up at the *car’s actual speed* rather than the *reference speed* v_{ref,i^*} keeps the reference spread short during startup acceleration; if the slow car were given a reference that already extends several metres ahead, the controller would chase a point it cannot physically reach in the horizon, biasing the solution toward unrealistic inputs. Once the car is up to speed the two choices coincide.

3 MPPI — Model Predictive Path Integral Control

3.1 Conceptual Overview

MPPI (Williams et al., 2017) is a **sampling-based** MPC variant rooted in stochastic optimal control and information theory. Instead of solving an optimisation problem, it runs K random *rollouts* of the vehicle model forward in time, scores each one with a cost function, and then computes the optimal control as a cost-weighted average of all the rollout inputs.

On the RoboRacer, at every 25 ms control step:

1. $K = 300$ random perturbation sequences are sampled.
2. Each is propagated through the bicycle model for $N = 20$ steps.
3. Each trajectory is scored: trajectories that stay close to the raceline receive high importance weight; trajectories that deviate are down-weighted.
4. The optimal input is a soft mixture of all sequences, dominated by the best-performing ones.
5. Only the first input of the mixture is applied to the car; the full sequence is shifted and reused as the warm start for the next step.

3.2 Discrete-Time Dynamics (Euler)

MPPI propagates an ensemble of states simultaneously using vectorised forward Euler integration:

$$X_{t+1}^{(k)} = X_t^{(k)} + v_t^{(k)} \cos \psi_t^{(k)} \Delta t \quad (5)$$

$$Y_{t+1}^{(k)} = Y_t^{(k)} + v_t^{(k)} \sin \psi_t^{(k)} \Delta t \quad (6)$$

$$\psi_{t+1}^{(k)} = \psi_t^{(k)} + \frac{v_t^{(k)}}{L} \tan \delta_t^{(k)} \Delta t \quad (7)$$

$$v_{t+1}^{(k)} = \text{clip}\left(v_t^{(k)} + a_t^{(k)} \Delta t, 0, v_{\max}\right) \quad (8)$$

The clip in the velocity update is a practical bound: it is non-differentiable, but MPPI never requires gradients through the dynamics.

3.3 Algorithm

Let $\mathbf{U} = [\mathbf{u}_0, \dots, \mathbf{u}_{N-1}] \in \mathbb{R}^{N \times 2}$ be the current nominal (warm-start) sequence and $\lambda > 0$ the temperature.

Step 1 – Sample perturbations.

$$\boldsymbol{\varepsilon}^{(k)} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \quad k = 1, \dots, K$$

with $\boldsymbol{\Sigma} = \text{diag}(\sigma_\delta^2, \sigma_a^2)$, $\sigma_\delta = 0.3 \text{ rad}$, $\sigma_a = 1.5 \text{ ms}^{-2}$. Note that σ_δ is comparable to $\delta_{\max} = 0.44 \text{ rad}$: by 1σ the samples already span most of the steering range, ensuring the corner-radius inputs needed on the figure-8 are present in the rollout cloud.

Step 2 – Form and clip perturbed sequences.

$$\mathbf{V}^{(k)} = \text{clip}\left(\mathbf{U} + \boldsymbol{\varepsilon}^{(k)}, \mathbf{u}_{\min}, \mathbf{u}_{\max}\right)$$

Step 3 – Roll out and accumulate cost. Starting from the current state $\mathbf{x}_0$, propagate each sequence and accumulate the running cost:

$$J^{(k)} = \sum_{t=0}^{N-1} c\left(\mathbf{x}_t^{(k)}, \mathbf{V}_t^{(k)}\right)$$

Step 4 – Compute importance weights. Subtract the minimum for numerical stability, then normalise:

$$\beta = \min_k J^{(k)}, \quad \tilde{w}_k = \exp\left(-\frac{J^{(k)} - \beta}{\lambda}\right), \quad w_k = \frac{\tilde{w}_k}{\sum_j \tilde{w}_j}$$

Low temperature λ concentrates weight on the best rollouts (greedy); high temperature spreads it more evenly (exploratory).

Step 5 – Update nominal sequence.

$$\mathbf{U}^* = \sum_{k=1}^K w_k \mathbf{V}^{(k)}$$

Step 6 – Apply and shift. Apply $\mathbf{u}_0^*$ to the car. Shift the sequence left by one step and zero-pad the tail to form the warm start for the next control iteration.

3.4 Cost Function

$$c(\mathbf{x}_t, \mathbf{u}_t) = W_{\text{CTE}} \underbrace{\|\mathbf{p}_t - \mathbf{p}_{\text{ref}}(t)\|_2}_{\text{cross-track error}} + W_h \underbrace{|\psi_t - \psi_{\text{ref}}(t)|_\pi}_{\text{heading error}} + W_v \underbrace{|v_t - v_{\text{ref}}(t)|}_{\text{speed error}} + W_s \underbrace{(\delta_t - \delta_{t-1})^2}_{\text{steering rate}}$$

where $|\cdot|_\pi$ denotes the wrapped absolute angle difference and $\mathbf{p}_{\text{ref}}(t)$, $\psi_{\text{ref}}(t)$, $v_{\text{ref}}(t)$ are looked up from the raceline at the projected arc position.

A subtle but important detail is that the steering term penalises the *rate* of change $(\delta_t - \delta_{t-1})^2$ rather than the absolute magnitude δ_t^2 . Penalising the magnitude would discourage holding the sustained $\delta \approx 0.21$ rad required to follow the figure-8 loops (radius $R \approx 1.5$ m, $\delta_{\text{steady}} = \arctan(L/R)$), causing systematic understeer. Penalising the rate suppresses bang-bang jitter without fighting steady cornering inputs. The previous control δ_{t-1} is seeded from the actual last applied input at step $t = 0$ rather than from zero, anchoring the rate term to physical continuity.

Weight	Value	Meaning
W_{CTE}	23.0	Penalise lateral deviation from the raceline
W_h	20.0	Penalise heading misalignment
W_v	0.5	Penalise speed error relative to v_{ref}
W_s	0.0005	Penalise steering rate $(\delta_t - \delta_{t-1})^2$

Other implementation parameters:

Parameter	Value	Meaning
K	300	Number of rollouts per control step
N	20	Horizon length (steps)
Δt	0.025 s	Prediction step size (matches main loop period)
λ	20.0	MPPI temperature
σ_δ	0.3 rad	Steering perturbation std
σ_a	1.5 m s ⁻²	Acceleration perturbation std

3.5 Warm-Start from Raceline Curvature

On the very first control iteration the nominal sequence $\mathbf{U}$ has no prior solve to inherit from. Initialising it to zero biases all rollouts toward straight-line motion, which catastrophically lags the controller through the first corner. Instead, the initial $\mathbf{U}$ is derived directly from the raceline geometry.

For each horizon step k , two arc-length positions are sampled, $s_k = s_{i^*} + kv_{\text{look}}\Delta t$ and $s_{k+1} = s_{i^*} + (k+1)v_{\text{look}}\Delta t$, with corresponding raceline headings ψ_k and ψ_{k+1} . Inverting the bicycle yaw-rate equation (3),

$$\dot{\psi} = \frac{v}{L} \tan \delta \implies \delta_k = \text{atan2}(L \cdot (\psi_{k+1} - \psi_k)_\pi, v_{\text{look}} \Delta t)$$

gives the steering input that would follow the raceline arc segment. The `atan2` form is used over `arctan` to avoid division-by-zero at small v_{look} . The acceleration component is left at zero; the speed controller catches up within the first second.

3.6 Velocity Tracking Change

Earlier iterations of MPPI used v_{ref,i^*} to set the lookahead spread. At startup this caused the reference to leap several metres ahead of a stationary car, producing rollouts that all tried to teleport rather than accelerate. The current code uses $v_{\text{look}} = \max(\hat{v}, 0.8)$, which scales the reference spread with actual progress. This and matching $\Delta t = 0.025\text{s}$ to the main loop period (it was previously 0.05s) were the two largest single-parameter improvements during development.

4 Nonlinear MPC

4.1 Conceptual Overview

Nonlinear MPC frames vehicle control as a **constrained optimisation problem** solved at every timestep. Rather than sampling random trajectories, it *solves* for the single best input sequence that minimises a quadratic cost subject to the vehicle dynamics and hard input/state constraints.

On the RoboRacer, at every 25 ms control step:

1. The current Vicon pose is fed to the `acados` solver as the initial condition.
2. The reference trajectory for the next N prediction steps is looked up from the raceline.
3. The solver performs one SQP-RTI step: it linearises the nonlinear dynamics around the warm-started trajectory, solves the resulting quadratic programme (QP) using HPIPM, and returns the optimal input.
4. Only the first input of the sequence is applied; the rest warm-start the next solve.

4.2 Continuous-Time Dynamics

Unlike MPPI, the MPC uses the continuous-time ODE (1)–(4) directly. `acados` integrates it numerically using explicit Runge–Kutta 4 (ERK4):

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\Delta t}{6} (\mathbf{f}_1 + 2\mathbf{f}_2 + 2\mathbf{f}_3 + \mathbf{f}_4)$$

where $\mathbf{f}_1, \dots, \mathbf{f}_4$ are the standard RK4 stage derivatives of $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$. This is more accurate than forward Euler and does not require the velocity clip; speed bounds are instead expressed

as explicit inequality constraints:

$$0 \leq v_k \leq v_{\max} \quad \forall k = 1, \dots, N$$

4.3 Optimal Control Problem

At time t with current state $\mathbf{x}_0$:

$$\begin{aligned} \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \quad & \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_N(\mathbf{x}_N) & (9) \\ \text{subject to} \quad & \mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k) & (\text{dynamics}) \\ & \mathbf{u}_k \in [\mathbf{u}_{\min}, \mathbf{u}_{\max}] & (\text{input bounds}) \\ & v_k \in [0, v_{\max}] & (\text{speed bound}) \\ & \mathbf{x}_0 = \mathbf{x}_{\text{current}} & (\text{initial condition}) \end{aligned}$$

where $F(\mathbf{x}_k, \mathbf{u}_k)$ denotes one ERK4 step of the bicycle ODE.

4.4 Cost Function

A Linear Least-Squares (LINEAR_LS) cost is used:

$$\ell(\mathbf{x}_k, \mathbf{u}_k) = \|\mathbf{y}_k - \mathbf{y}_{\text{ref},k}\|_{\mathbf{W}}^2 = (\mathbf{y}_k - \mathbf{y}_{\text{ref},k})^\top \mathbf{W} (\mathbf{y}_k - \mathbf{y}_{\text{ref},k})$$

with the residual vector

$$\mathbf{y}_k = \begin{pmatrix} X_k \\ Y_k \\ \psi_k \\ v_k \\ \delta_k \\ a_k \end{pmatrix}, \quad \mathbf{y}_{\text{ref},k} = \begin{pmatrix} X_{\text{ref},k} \\ Y_{\text{ref},k} \\ \psi_{\text{ref},k} \\ v_{\text{ref},k} \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{W} = \text{diag}(W_{\text{CTE}}, W_{\text{CTE}}, W_h, W_v, W_s, W_a)$$

The zero reference for δ and a means the solver penalises control effort (regularisation), keeping the trajectory smooth. The terminal cost uses the same structure with $\mathbf{W}_N = \text{diag}(W_{\text{CTE}}, W_{\text{CTE}}, W_h, W_v)$.

Note that LINEAR_LS penalises the *absolute magnitude* δ_k^2 rather than the rate $(\delta_k - \delta_{k-1})^2$ used in MPPI. Encoding a rate penalty cleanly in LINEAR_LS would require augmenting the state with the previous control; in practice the RTI warm-start provides enough temporal smoothness on its own that this has not been necessary.

Weight	Value	Meaning
W_{CTE}	23.0	Position tracking (applied to both X and Y residuals)
W_h	20.0	Heading tracking
W_v	0.5	Speed tracking
W_s	1.0	Steering regularisation (absolute magnitude)
W_a	0.1	Acceleration regularisation

Other implementation parameters:

Parameter	Value	Meaning
N	20	Horizon length (steps)
Δt	0.025 s	Prediction step (matches main loop period)
QP solver	HPIPM PC	Partial-condensing interior point
NLP solver	SQP-RTI	One linearisation per control step
Integrator	ERK4	Explicit Runge–Kutta 4

4.5 Reference Heading Unwrapping

The continuous-time dynamics do not wrap ψ — a discontinuity at $\pm\pi$ would break the linearisation that SQP-RTI relies on. The raceline’s ψ_{ref} values, however, are stored in $[-\pi, \pi]$ from `atan2`, and the figure-8 contains a $\pm\pi$ wrap roughly halfway through the loop. Feeding a wrapped reference into a non-wrapping state model produces a huge residual at the wrap point.

The MPC therefore **unwraps** the reference incrementally: starting from $\tilde{\psi}_0 = \psi_{\text{state}}$, each subsequent reference is built as

$$\tilde{\psi}_{k+1} = \tilde{\psi}_k + (\psi_{\text{ref},k+1} - \tilde{\psi}_k)_{\pi}$$

where $(\cdot)_{\pi}$ wraps an angle into $[-\pi, \pi]$. This produces a continuous reference $\tilde{\psi}_k$ in $\mathbb{R}$ that stays close to the predicted ψ_k trajectory throughout the horizon.

4.6 Warm-Start from Raceline Curvature

The same raceline-curvature initialisation used in MPPI is applied to the MPC solver on its first iteration, before any previous solve exists to warm-start from. Stage inputs are seeded with $\delta_k = \text{atan2}(L \Delta\psi_k, v_{\text{look}}\Delta t)$ and $a_k = 0$; stage states are seeded by walking along the raceline waypoints with the unwrapped reference heading. Acados then propagates from this guess in subsequent solves via internal warm-starting.

4.7 SQP Real-Time Iteration (SQP-RTI)

Solving the full NLP (9) to convergence at 40 Hz is infeasible for large horizons. The **Real-Time Iteration** (Diehl et al., 2005) scheme replaces the full NLP solve with a single SQP step per control cycle.

At each step, the NLP is linearised around the current warm-start trajectory $(\bar{\mathbf{X}}, \bar{\mathbf{U}})$ to obtain a **quadratic programme**:

$$\begin{aligned} \min_{\Delta \mathbf{U}} \quad & \frac{1}{2} \Delta \mathbf{U}^{\top} \mathbf{H} \Delta \mathbf{U} + \mathbf{g}^{\top} \Delta \mathbf{U} & (10) \\ \text{subject to} \quad & \mathbf{C} \Delta \mathbf{U} = \mathbf{c} & (\text{linearised dynamics}) \\ & \mathbf{u}_{\min} - \bar{\mathbf{U}} \leq \Delta \mathbf{U} \leq \mathbf{u}_{\max} - \bar{\mathbf{U}} & (\text{input bounds}) \end{aligned}$$

where $\mathbf{H}$ is the Gauss–Newton Hessian approximation and $\mathbf{g}$ the gradient of the cost. The QP is solved by **HPIPM** (partial condensing), and the solution $\Delta \mathbf{U}^*$ is added to the warm start.

Because the initial state changes only slightly between control steps (the car has moved 25 ms worth), the warm-started trajectory is already close to the new optimum and one SQP step is sufficient to track it accurately.

5 PID Baseline (Stanley-style)

5.1 Conceptual Overview

The PID controller is the classical baseline against which MPPI and MPC are benchmarked. Unlike the two predictive controllers, it has no internal model and no horizon: it produces a steering command directly from the *current* cross-track and heading errors. Structurally it is a *Stanley-style* steering law, augmented with an integral term:

$$\delta = -(K_h \psi_e + K_e e + K_{\dot{e}} \dot{e} + K_f \int e dt)$$

The overall negative sign converts a positive (left-of-track) lateral error into a right-steer command, given the sign convention of δ in the bicycle model. All four gains are kept positive; flipping the sign of any single one would reverse the corresponding correction direction.

5.2 Signed Cross-Track Error

The plain Euclidean distance $\|\mathbf{p} - \mathbf{p}_{\text{ref}}\|$ used by MPPI and MPC for cost is unsigned and cannot indicate which side of the track the car is on. PID needs a **signed** CTE to know which direction to correct. Decomposing the position offset along the raceline’s local frame,

$$\hat{\mathbf{t}}_{\text{ref}} = (\cos \psi_{\text{ref}}, \sin \psi_{\text{ref}}), \quad \hat{\mathbf{n}}_{\text{ref}} = (-\sin \psi_{\text{ref}}, \cos \psi_{\text{ref}}),$$

gives the signed CTE as the projection of the offset onto the left-normal:

$$e = (\mathbf{p} - \mathbf{p}_{\text{ref}}) \cdot \hat{\mathbf{n}}_{\text{ref}} = -\sin \psi_{\text{ref}} (X - X_{\text{ref}}) + \cos \psi_{\text{ref}} (Y - Y_{\text{ref}})$$

By convention $e > 0$ when the car is to the *left* of the track.

5.3 Derivative and Integral Terms

The derivative term $\dot{e}$ is approximated by a finite difference between successive control steps,

$$\dot{e} \approx \frac{e_t - e_{t-1}}{\Delta t},$$

with Δt clamped to $[10^{-6}, 0.2]$ s to prevent first-step spikes and stale-frame outliers from dominating the derivative.

The integral term $\int e dt$ is implemented as a discrete sum with **clamping anti-windup**: the running integral is bounded so that the integral contribution to δ cannot exceed a configurable cap I_{max} , and the integral is frozen entirely while the total steering command is saturated at $\pm \delta_{\text{max}}$. The combination prevents the integrator from accumulating during corner saturation and then overshooting on exit.

5.4 Heading Error Term and Stanley Analogue

The Stanley controller in its original form is

$$\delta = \psi_e + \arctan\left(\frac{k e}{v}\right).$$

For small CTE the second term linearises to $k e/v$, motivating $K_e \approx k/v$ as a starting gain. At $v \approx 4.5$ m/s with $k = 2$ this gives $K_e \approx 0.44$ rad/m. The heading term has gain $K_h = 1.0$ (unit gain) so that small heading errors map directly to proportional steering corrections; the closed-loop heading time constant of this term is $\tau_h = L/(vK_h)$, which at $v = 4.5$ m/s evaluates to $\tau_h \approx 72$ ms.

5.5 Parameters

Default tuning (modifiable from the CLI):

Gain	Value	Meaning
K_h	0.6	Heading-error P gain (rad/rad)
K_e	0.6	Cross-track P gain (rad/m)
$K_{\dot{e}}$	0.05	Cross-track D gain (rad·s/m)
K_f	0.0	Cross-track I gain (rad/(m·s))
$I_{\max}$	0.10 rad	Anti-windup cap on integral contribution

The integral gain is left at zero by default: the figure-8 is symmetric and on a well-tuned PD the steady-state offset is negligible. K_f is only introduced when persistent one-sided drift is observed (e.g. from chassis misalignment).

5.6 Tuning Procedure

Gains are tuned in a fixed order to isolate one closed-loop effect at a time:

1. Set $K_e, K_{\dot{e}}, K_f$ to zero; tune K_h alone on a straight until heading error settles below 0.1 rad with no oscillation.
2. Add K_e to remove lateral drift; expect to revisit K_h once K_e is active because the two terms partially compensate each other.
3. Add $K_{\dot{e}}$ if the closed loop is now oscillatory but non-divergent. D-gain is bounded above by measurement-noise amplification: roughly $K_{\dot{e}} < K_e/2$ at 40 Hz.
4. Add K_f only if a persistent one-sided offset remains.

6 Throttle Control

Neither controller directly commands a wheel torque. Both use a **feedforward + proportional speed controller** that maps the raceline reference velocity to a dimensionless throttle integer (0–200):

$$\tau = \text{clip}(K_{\text{ff}} v_{\text{ref}} + K_p (v_{\text{ref}} - \hat{v}), 0, \tau_{\max})$$

where $\hat{v}$ is the speed estimate from Vicon position differences (EMA-smoothed), $K_{\text{ff}} = 20$ and $K_p = 5$ are default gains. The MPPI acceleration output a and the MPC acceleration command a^* are not used for throttle for four compounding reasons:

1. **No calibrated actuator model.** Both controllers output a in m s^{-2} , but the hardware input is a dimensionless throttle integer sent to an ESC. The mapping throttle $\rightarrow$ torque $\rightarrow$ acceleration varies with speed, battery voltage, and surface friction. Without empirical system identification a direct $a \rightarrow \tau$ conversion is unreliable.
2. **Noisy velocity estimate.** $\hat{v}$ is computed from finite differences of Vicon positions and is EMA-smoothed but still noisy. That noise enters the MPPI/MPC initial state, propagates through all rollouts or linearisations, and produces a noisy a command. A simple proportional controller on $v_{\text{ref}} - \hat{v}$ is far more forgiving of this noise than closing the throttle loop through the optimiser.
3. **Speed profile already solved offline.** The raceline CSV encodes a pre-optimised speed profile (slow for tight corners, fast on straights). Re-optimising speed online adds complexity without benefit when the reference profile is already well-designed; the controller need only track it.
4. **Decoupled tuning.** At low slip angles, lateral and longitudinal dynamics interact weakly. Keeping the speed loop separate means K_{ff} and K_p can be tuned independently of the MPPI hyper-parameters or MPC weights, making both easier to diagnose.

The steering servo command (integer 120–880) is derived from the controller’s steering angle output in both cases:

$$\tau_{\delta} = \text{clip}(\tau_{\text{center}} + K_{\text{srv}} \delta + \tau_{\text{trim}}, \tau_{\text{min}}, \tau_{\text{max}})$$

with $\tau_{\text{center}} = 512$, $K_{\text{srv}} = 1200$ servo-units/rad, $\tau_{\text{trim}} = 20$.

7 Future Model Improvements

7.1 Dynamic Bicycle Model

The kinematic model assumes zero tire slip, which is only valid when lateral acceleration $a_y \ll \mu g$. At higher speeds the tires develop meaningful slip angles and the kinematic model underestimates the required steering.

The **dynamic bicycle model** adds lateral velocity v_y and yaw rate $r = \dot{\psi}$ as states, and replaces the kinematic steering constraint with force balance equations:

$$\mathbf{x} = \begin{pmatrix} X \\ Y \\ \psi \\ v_x \\ v_y \\ r \end{pmatrix}, \quad m\dot{v}_y = F_{y,f} + F_{y,r} - mv_x r, \quad I_z \dot{r} = L_f F_{y,f} - L_r F_{y,r}$$

where m is vehicle mass, I_z yaw moment of inertia, L_f/L_r are distances from CG to front/rear axles, and $F_{y,f}$, $F_{y,r}$ are lateral tire forces. For a linear tire model: $F_{y,f} = -C_f\alpha_f$, $F_{y,r} = -C_r\alpha_r$, with slip angles

$$\alpha_f = \delta - \arctan\left(\frac{v_y + L_f r}{v_x}\right), \quad \alpha_r = -\arctan\left(\frac{v_y - L_r r}{v_x}\right)$$

and cornering stiffnesses C_f , C_r identified from vehicle tests.

7.2 Pacejka “Magic Formula” Tire Model

For cornering beyond the linear regime (slipping or near-limit driving), the linear tire force model fails. The **Pacejka magic formula** (Pacejka, 2012) captures the full nonlinear force–slip curve:

$$F_y(\alpha) = D \sin\left[C \arctan\left(B\alpha - E(B\alpha - \arctan(B\alpha))\right)\right]$$

where B (stiffness factor), C (shape factor), D (peak value), and E (curvature factor) are tire-specific parameters. At small slip angles the formula reduces to the linear model ($F_y \approx BCD\alpha$, so $C_{\text{eff}} = BCD$), ensuring consistency with the linear model.

Replacing the linear tire forces in the dynamic bicycle model with the Pacejka formula gives a **quarter-car + Pacejka** model that is accurate from low-speed cruising through near-limit cornering. Both MPPI and the MPC formulation can accommodate it without structural changes: MPPI requires only that the forward model be evaluable; MPC requires additionally that F_y is differentiable, which the smooth Pacejka formula satisfies.

8 Experimental Results

All three controllers are benchmarked on the same figure-8 raceline (loop radius $R \approx 1.5$ m, total arc length ≈ 17.3 m, constant reference speed $v_{\text{ref}} = 4.5 \text{ m s}^{-1}$) over 10 laps each. Errors are accumulated per control step from the lap start to the lap end and reported as lap-averaged RMS values.

8.1 Metrics

For each lap ℓ , with T_ℓ samples:

$$\text{RMS}_e^{(\ell)} = \sqrt{\frac{1}{T_\ell} \sum_{t=1}^{T_\ell} e_t^2}, \quad \text{RMS}_{\psi_e}^{(\ell)} = \sqrt{\frac{1}{T_\ell} \sum_{t=1}^{T_\ell} \psi_{e,t}^2}, \quad \text{RMS}_{v_e}^{(\ell)} = \sqrt{\frac{1}{T_\ell} \sum_{t=1}^{T_\ell} (v_t - v_{\text{ref},t})^2}.$$

Aggregate metrics are the means and standard deviations over $\ell = 1, \dots, 10$.

8.2 Results

Controller	$ e _{\text{RMS}}$ (m)	$ \psi_e _{\text{RMS}}$ (rad)	$ v_e _{\text{RMS}}$ (m/s)	Notes
PID	—	—	—	Baseline
MPPI	—	—	—	300 rollouts, $\lambda = 20$
MPC	—	—	—	SQP-RTI, ERK4

Relative differences (with PID as the reference):

Controller	$\Delta e $ vs PID	$\Delta \psi_e $ vs PID	$\Delta v_e $ vs PID
MPPI	—	—	—
MPC	—	—	—

8.3 Discussion

(To be filled in once the 10-lap runs are collected.) Expected qualitative findings, to be confirmed by the data:

- **PID** should have the largest $|e|_{\text{RMS}}$, especially through corner entries and exits, because it reacts only to current error and cannot anticipate the curvature change.
- **MPPI** should outperform PID by virtue of horizon-aware planning, but show non-zero step-to-step variation in δ because the weighted-average update is stochastic. Steering RMS should be slightly higher than MPC.
- **MPC** should produce the smoothest steering trace and the smallest tracking errors, at the cost of compile-time complexity and a (one-time) solver build step.
- All three should track v_{ref} similarly, since the longitudinal channel is governed by the same feedforward-plus-P throttle controller in every case.